JAVA • CORE CONCEPT NOTES

Multithreading

A complete, exam-focused study guide with code, diagrams & revision sheet

Topics Covered

1. Multithreading Concept
2. Thread Life Cycle `(New, Runnable, Running, Blocked, Waiting, Timed Waiting, Terminated)`

Prepared for quick understanding and last-minute revision

TOPIC 1

Multithreading Concept

1.1 What is Multithreading?

A **thread** is the smallest unit of execution within a process — sometimes called a *lightweight process*. **Multithreading** is a Java feature that allows concurrent execution of two or more parts (threads) of the same program, so that maximum utilization of the CPU is achieved. Each thread runs in parallel (logically) and shares the same memory area of the process it belongs to.

In Java, multithreading is supported at the language level through the `Thread` class and the `Runnable` interface, both present in the `java.lang` package.

THREAD CREATION HIERARCHY

```
java.lang.Object
  `-- Thread (implements Runnable – has run(), start(), sleep() ...)

java.lang.Runnable (functional interface)
  `-- implemented by any class that defines run()
```

1.2 Need and Purpose of Multithreading

- **Better CPU utilization:** While one thread waits (e.g., for I/O), another thread can use the CPU.
- **Simultaneous tasks:** Enables a program to perform multiple operations at once (e.g., downloading a file while updating the UI).
- **Faster, responsive applications:** Long-running tasks do not freeze the entire application.
- **Efficient resource sharing:** Threads of the same process share memory, so communication between them is cheaper than between separate processes.

1.3 Process vs Thread

Point	Process	Thread
Definition	An independent, self-contained program in execution	A single execution path within a process
Memory	Has its own separate memory space	Shares memory of the parent process
Communication	Costly (Inter-Process Communication needed)	Easy and fast (shared memory)
Creation overhead	Heavyweight — slow to create/terminate	Lightweight — fast to create/terminate
Dependency	Independent of other processes	Depends on the process that owns it

Program Example: Running this program starts **one process**. The labelled output below shows that process clearly, and the two separate threads — `main` and `Thread-0` — running inside it.

```java
public class ProcessVsThreadDemo {
    public static void main(String[] args) {
        // This running program = ONE PROCESS
        System.out.println("PROCESS : ProcessVsThreadDemo (this program)");

        // The thread executing main() right now
        System.out.println("THREAD  : " + Thread.currentThread().getName());

        // A second thread -- same process, different thread
        Thread t1 = new Thread(() -> {
            System.out.println("THREAD  : " + Thread.currentThread().getName());
        });
        t1.start();
    }
}
```

Sample Output:

```
PROCESS : ProcessVsThreadDemo (this program)
THREAD  : main
THREAD  : Thread-0
```

Observation: There is only **one PROCESS** line — the single running program — but **two THREAD** lines (`main` and `Thread-0`), showing that one process can contain and run multiple threads.

EXAM POINT
A process can contain multiple threads, but a thread cannot exist without a parent process. All threads of a process share the same code, data, and heap, but each has its own stack, program counter, and register set.

1.4 Characteristics and Advantages

- Threads share the same address space, reducing memory overhead compared to multiple processes.
- Context-switching between threads is faster than between processes.
- Improves performance of applications running on multi-core processors.
- Allows overlapping of computation and I/O (e.g., one thread computes while another reads a file).
- Makes programs more responsive, especially in GUI and server applications.

1.5 Basic Working of Multiple Threads

On a single CPU core, the operating system's thread scheduler rapidly switches the CPU between ready threads *(it switches which thread gets to use the one CPU — via **time-slicing / context switching**)*, so the threads appear to run simultaneously (this is *concurrency*). On multi-core processors, threads may genuinely execute at the same instant on different cores (*true parallelism*). In both cases, all threads within a single process share the process's code, data segment and heap memory, while each maintains its own stack, program counter and register set.

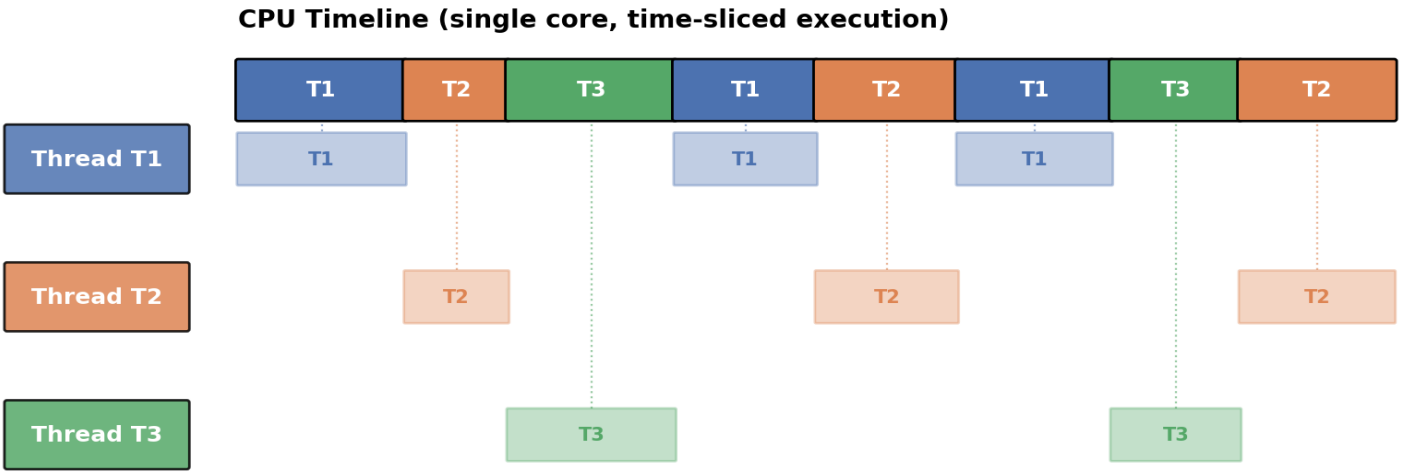


Fig 1.1 — Multithreading execution concept: threads take turns on the CPU (concurrency)

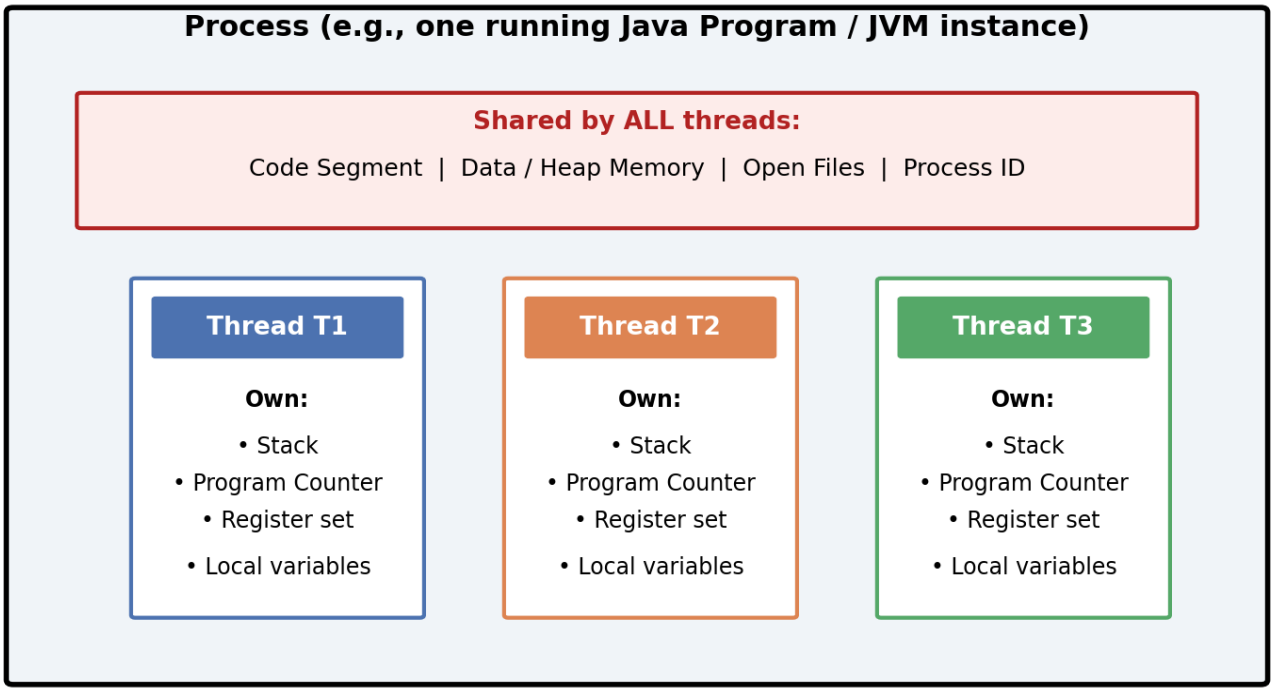


Fig 1.2 — Multiple threads running within a single process, sharing common resources

1.6 Simple Java Example

Java provides two main ways to create a thread: by **extending the Thread class** or by **implementing the Runnable interface**. The example below extends `Thread` and overrides its `run()` method — the code inside `run()` is what executes in the new thread.

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName()
                               + " : count = " + i);
        }
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.setName("Thread-A");
        t2.setName("Thread-B");
        t1.start();   // starts new thread, calls run()
        t2.start();   // starts another thread, calls run()
    }
}
```

Explanation: Calling `start()` creates a new thread of execution and internally invokes `run()` on it. Calling `run()` directly (without `start()`) would NOT create a new thread — it would simply execute like a normal method call on the current thread. Because both threads run concurrently, the scheduler decides the order, so output lines from Thread-A and Thread-B may interleave unpredictably.

Sample Output (order may vary on each run):

```
Thread-A : count = 1
Thread-B : count = 1
Thread-A : count = 2
Thread-B : count = 2
Thread-A : count = 3
Thread-B : count = 3
```

COMMON MISTAKE
`start()` vs `run()` — `start()` creates a new call stack and a genuinely new thread; `run()` alone just runs the code on the calling thread, with no multithreading benefit.

TOPIC 2

Thread Life Cycle

2.1 Definition

The **Thread Life Cycle** describes the various states a thread passes through during its lifetime, from creation to termination. In Java, these states are represented by the `Thread.State` enum. A thread moves between states based on method calls (such as `start()`, `sleep()`, `wait()`) and scheduler actions.

2.2 Thread States

State	Description
New	Thread object is created (e.g., 'new Thread()') but start() has not yet been called. No system resources for execution are allocated yet.
Runnable	start() has been called; the thread is ready to run and is waiting for the CPU to be scheduled by the thread scheduler.
Running	The thread scheduler has selected this thread and the CPU is actively executing its run() method code.
Blocked	The thread is temporarily inactive because it is waiting for a lock/monitor held by another thread to become free (e.g., trying to enter a synchronized block).
Waiting	The thread is waiting indefinitely for another thread to perform a particular action — e.g., it called wait() and is waiting until notified via notify() / notifyAll().
Timed Waiting	The thread waits for a specified period, e.g., due to sleep(ms) or join(ms), and resumes automatically after the time elapses.
Terminated (Dead)	The thread has completed executing run() (normal exit) or was stopped abnormally. Once terminated, it cannot be restarted.

2.3 State Transition Diagram

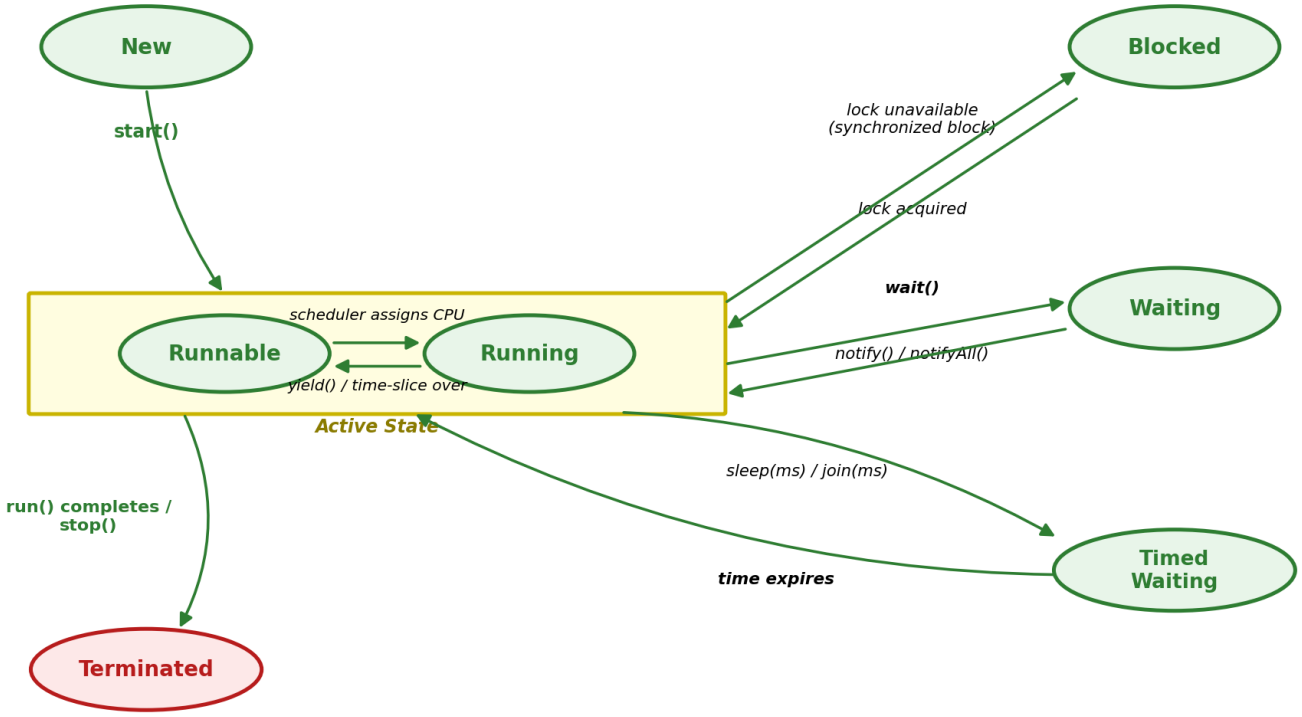


Fig 2.1 — Complete Thread Life Cycle: states and transition events

2.4 Conditions / Events Causing Transitions

Transition	Triggering Event / Method
New → Runnable	start() is called on the thread object
Runnable → Running	Thread scheduler selects the thread and allocates CPU time
Running → Runnable	Time slice expires, or yield() is called (thread gives up CPU voluntarily)
Running → Blocked	The thread tries to enter a synchronized block/method but the lock is held by another thread
Blocked → Runnable	The lock becomes available and is acquired by this thread
Running → Waiting	wait() is called (thread waits indefinitely for a signal)
Waiting → Runnable	notify() / notifyAll() is called by another thread
Running → Timed Waiting	sleep(ms) or join(ms) is called with a specified duration
Timed Waiting → Runnable	The specified sleep/join time expires
Running → Terminated	run() method completes normally, or the thread is stopped abnormally

EXAM POINT
A thread in the **New** state has no thread of execution yet — calling any method other than start() (or stop()) on it is invalid. Also, a thread that has entered **Terminated** state cannot be restarted by calling start() again — doing so throws an `IllegalThreadStateException`.

2.5 Java-Oriented Explanation with Example

The current state of any Java thread object can be checked using `getState()`, which returns a value from the `Thread.State` enum: `NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED`.

```
class DemoThread extends Thread {
    public void run() {
        try {
            Thread.sleep(1000);   // moves to TIMED_WAITING
        } catch (InterruptedException e) { }
    }
}

public class LifeCycleDemo {
    public static void main(String[] args) throws InterruptedException {
        DemoThread t = new DemoThread();
        System.out.println("After creation   : " + t.getState());   // NEW
        t.start();
        System.out.println("After start()    : " + t.getState());   // RUNNABLE
        Thread.sleep(200);
        System.out.println("While sleeping   : " + t.getState());   // TIMED_WAITING
        t.join();  // main thread waits for t to finish
        System.out.println("After completion : " + t.getState());   // TERMINATED
    }
}
```

Expected Output:

```
After creation   : NEW
After start()    : RUNNABLE
While sleeping   : TIMED_WAITING
After completion : TERMINATED
```

NOTE
The RUNNABLE state in Java's Thread.State enum covers both 'ready to run' and 'currently running' — Java does not expose a separate 'Running' enum value; the OS-level distinction between Runnable and Running is conceptual, used for understanding the life cycle diagram.

QUICK REVISION

Key Facts & Definitions

Key Definitions

- **Thread:** Smallest unit of execution within a process; a lightweight sub-process.
- **Multithreading:** Concurrent execution of multiple threads within a single process to maximize CPU utilization.
- **Thread Life Cycle:** The sequence of states (New → Runnable → Running → Blocked / Waiting / Timed Waiting → Terminated) a thread passes through.

Thread States (Java Thread.State enum)

NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING <-> RUNNABLE) → TERMINATED

Key Method → Transition Map

Method	Effect
start()	New → Runnable
run()	Contains the code executed by the thread (called automatically by start())
sleep(ms)	Running → Timed Waiting (for fixed duration)
wait()	Running → Waiting (until notify()/notifyAll())
join()	Calling thread waits until the target thread terminates
yield()	Running → Runnable (gives other threads a chance)

Important Facts for Exam

- Threads of the same process share code, data and heap, but each has its own stack, program counter and registers.
- start() creates a new call stack; calling run() directly does NOT create a new thread.
- A terminated thread cannot be restarted — start() on it throws IllegalThreadStateException.
- Context switching between threads is cheaper than between processes because memory is shared.
- On a single-core CPU, multithreading gives concurrency (time-sliced); on multi-core CPUs, it can give true parallelism.